

MathScript

Operator Precedence and Function Evaluation

A code-based study guide for your viva

This guide explains two questions from the actual MathScript implementation:

1. How does the compiler decide operator precedence and associativity?
2. How do `sqrt`, `pow`, and the other built-in functions work, from source text to result?

The explanations use simple English, short code extracts, expression trees, worked calculations, and answers you can say aloud. The source files are listed beside the relevant explanations so you know what to open during a demonstration.

Code snapshot: MathScript source reviewed on 12 September 2026.

Reading plan: First read Chapters 1–4. Then rehearse the demonstration in Chapter 7 and the questions in Chapter 8. Use the appendix when you need the complete evaluator.

Contents

1	The whole idea before the details	3
1.1	What MathScript does	3
1.2	Five useful words	3
2	Operator precedence: the exact mechanism	5
2.1	The five declarations to show your professor	5
2.2	Why the grammar needs these declarations	5
2.3	Shift and reduce, explained simply	6
2.4	Reading a Bison action	6
2.5	Unary minus and the special UMINUS symbol	7
3	Worked precedence examples	8
3.1	Example 1: multiplication before addition	8
3.2	Example 2: parentheses change the tree	8
3.3	Example 3: left associativity	9
3.4	Example 4: right-associative exponentiation	9
3.5	Example 5: unary minus versus exponentiation	9
3.6	Example 6: comparisons have lower precedence	9
3.7	A real limitation: implicit multiplication and powers	9
4	How sqrt works, from text to answer	11
4.1	Step 1: recognize the tokens	11
4.2	Step 2: recognize a function call	11
4.3	Step 3: store the function-call node	11
4.4	Step 4: dispatch to the function evaluator	12
4.5	Step 5: check the argument count	12
4.6	Step 6: check the domain and call the C library	12
4.7	What the project does not implement inside sqrt	13
5	Every supported function and related rule	14

5.1	Function reference	14
5.2	abs, sin, cos, tan, and log	14
5.3	pow: evaluate two arguments	15
5.4	How argument lists work	15
5.5	Numbers, variables, and constants	15
6	Recursion, errors, and code generation	17
6.1	One complete nested example	17
6.2	Why ok is a pointer	17
6.3	Why quiet exists	18
6.4	The four error classes	18
6.5	Assignment is different from equality	18
6.6	Square root versus imaginary equation roots	18
6.7	What gen demonstrates	18
7	A demonstration you can rehearse	20
7.1	A five-minute sequence	20
7.2	Expected answers for the successful calculations	21
7.3	A quick practice exercise	21
8	Short viva questions and answers	22
A	The actual evaluator source	25
B	Source-file revision map	28

Chapter 1

The whole idea before the details

1.1 What MathScript does

MathScript reads a line, understands its structure, and evaluates, plots, or solves it according to the selected mode. Flex supplies the lexical analyzer; Bison generates the parser; the remaining logic is written in C.

Stage	Responsibility in this project
Lexer	<code>src/lexer.l</code> : turn characters into tokens.
Parser	<code>src/parser.y</code> : recognize grammar rules and construct an AST.
AST	<code>src/ast.h</code> , <code>src/ast.c</code> : store the expression as a tree.
Evaluation	<code>src/eval.c</code> : recursively calculate values and perform semantic and mathematical checks.
Mode handling	<code>src/calc.c</code> , <code>src/graph.c</code> , <code>src/solver.c</code> : interpret the statement for the active mode.
Printed IR	<code>src/ir.c</code> : walk the AST and print three-address code (TAC).
C generation	<code>src/codegen.c</code> : walk the AST to produce a separate C program for <code>gen</code> .

An important implementation detail: normal calculation uses the AST evaluator directly. TAC is a separate representation printed from the AST. The current C generator also traverses the AST; it does not read the printed TAC. Both outputs express the operations in the same parsed tree.

Say it simply: The lexer recognizes the pieces, the parser decides their structure, and the evaluator calculates the value of that structure.

1.2 Five useful words

Token A recognized piece such as a number, identifier, or plus sign.

Grammar rule A valid pattern, such as an expression followed by `+` and another expression.

AST Abstract syntax tree: a compact tree of the meaningful operations.

Precedence Which operator binds more tightly when different operators occur together.

Associativity How repeated operators at the same precedence level group.

For $2 + 3 * 4$, precedence produces the tree for $2 + (3 \times 4)$. For $10 - 3 - 2$, associativity produces the tree for $(10 - 3) - 2$.

Chapter 2

Operator precedence: the exact mechanism

2.1 The five declarations to show your professor

Open `src/parser.y`. These are the current declarations, in their actual order:

```
%left '<' '>' GE LE EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS
%right '^'
```

Read from top to bottom: later declarations have higher precedence. Operators on the same line have equal precedence. GE, LE, EQ, and NE represent `>=`, `<=`, `==`, and `!=`.

Level	Operators	Grouping rule
1, lowest	<code><</code> <code>></code> <code>>=</code> <code><=</code> <code>==</code> <code>!=</code>	Left associative
2	<code>+</code> <code>-</code>	Left associative
3	<code>*</code> <code>/</code>	Left associative
4	Unary <code>-</code>	Uses the artificial precedence symbol UMINUS
5, highest declared	<code>^</code>	Right associative

Parentheses and function calls are handled by their grammar productions. They do not need an additional line in this operator table.

Say it simply: I declare precedence in Bison from lowest to highest. Multiplication is below exponentiation but above addition. The parser uses these declarations when constructing the AST.

2.2 Why the grammar needs these declarations

The expression grammar includes:

```
expr : expr '+' expr { $$ = ast_binop('+', $1, $3); }
      | expr '*' expr { $$ = ast_binop('*', $1, $3); }
```

```
;
```

This is an extract, not the complete grammar. Without additional precedence information, the pattern `expr operator expr` allows more than one structure for `2 + 3 * 4`:

$$(2 + 3) \times 4 = 20, \quad 2 + (3 \times 4) = 14.$$

The declarations resolve this ambiguity. Bison generates parser tables from the grammar and declarations when the project is built.

2.3 Shift and reduce, explained simply

Suppose the parser has recognized `2 + 3` and the next token is `*`. It has two possible actions:

- **Reduce:** treat `2 + 3` as a completed expression now.
- **Shift:** read the next token and continue building the expression.

Because `*` has higher precedence than `+`, Bison chooses to shift in this conflict. The multiplication becomes part of the right operand of the addition.

In a shift/reduce conflict where both precedences are known:

1. Higher token precedence: shift.
2. Higher rule precedence: reduce.
3. Equal precedence with `%left`: reduce.
4. Equal precedence with `%right`: shift.

Normally a rule receives the precedence of its last terminal symbol. `%prec` can explicitly replace the rule's precedence.

2.4 Reading a Bison action

```
expr : expr '+' expr { $$ = ast_binop('+', $1, $3); }
```

- `$1` is the value of the first item: the left expression's AST.
- `$2` corresponds to the plus token; this action does not use its value.
- `$3` is the right expression's AST.
- `$$` is the AST value produced by the entire rule.
- `ast_binop` creates an operation node with those two children.

The action constructs a tree; it does not add the numbers yet.

2.5 Unary minus and the special UMINUS symbol

```
%right UMINUS
%right '^'

/* In the expression rules: */
| '-' expr %prec UMINUS { $$ = ast_uminus($2); }
```

The same character `-` appears in subtraction and negation. `%prec UMINUS` gives the negation production a different precedence from binary subtraction. `UMINUS` is an artificial precedence symbol; the user does not type it and the lexer does not return it for input text.

The current code uses `%right UMINUS`, which is accepted by the older Bison installed on this machine. Its purpose here is to assign the unary rule its level. Exponentiation is declared later, so it binds more tightly:

$$-2^2 = -(2^2) = -4, \quad (-2)^2 = 4.$$

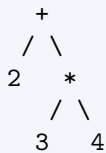
Say it simply: Unary minus uses an explicit rule precedence. It is higher than multiplication but lower than power, so minus two squared becomes minus four.

Chapter 3

Worked precedence examples

3.1 Example 1: multiplication before addition

Input: $2 + 3 * 4$.



1. The parser groups the multiplication on the right.
2. The evaluator obtains 2 from the left leaf.
3. It evaluates the right subtree: $3 \times 4 = 12$.
4. It calculates $2 + 12 = 14$.

Printed TAC:

```
t1 = 3 * 4
t2 = 2 + t1
RESULT = t2
```

Precedence determines the grouping. It does not mean the evaluator always visits the highest-precedence token first: this evaluator visits the left child before the right child, following the already constructed tree.

3.2 Example 2: parentheses change the tree

Input: $(2 + 3) * 4$. The grammar rule is:

```
| '(' expr ')' { $$ = $2; }
```

The enclosed expression becomes the operand. No extra parentheses node is needed in this AST. The tree has multiplication at its root and addition as its left child. Therefore $(2 + 3) \times 4 = 20$.

3.3 Example 3: left associativity

$$10 - 3 - 2 = (10 - 3) - 2 = 5.$$

Because subtraction is left associative, this is not $10 - (3 - 2) = 9$. Likewise:

$$20/5 \times 2 = (20/5) \times 2 = 8.$$

Multiplication and division have the same precedence. Multiplication does not always happen before division; their shared left associativity decides this case.

3.4 Example 4: right-associative exponentiation

Input: 2^{3^2} .

$$2^{3^2} = 2^9 = 512.$$

The right power groups first. The alternative $(2^3)^2$ would give 64.

```
t1 = 3 ^ 2
t2 = 2 ^ t1
RESULT = t2
```

3.5 Example 5: unary minus versus exponentiation

```
Input: -2^2
Tree:  UMINUS(BINOP('^', NUM(2), NUM(2)))

Printed TAC:
t1 = 2 ^ 2
t2 = 0 - t1
RESULT = t2
```

The AST represents unary negation explicitly. TAC expresses that negation as subtraction from zero. Both give -4 . A negative exponent also works: 2^{-3} represents $2^{-3} = 0.125$.

3.6 Example 6: comparisons have lower precedence

$2 + 3 > 4$ groups as $(2 + 3) > 4$, which is true. The evaluator represents true as 1 and false as 0, both returned as numeric values.

Chained comparisons are left associative in this language. For example, $3 < 2 < 1$ groups as $(3 < 2) < 1$: first 0, then $0 < 1$, so the result is 1. It is not a mathematical chained inequality.

3.7 A real limitation: implicit multiplication and powers

The grammar has special rules for `NUMBER IDENTIFIER` and `NUMBER '(' expr ')'`. These support $3x$ and $3(x+1)$. However, the current grammar reduces $3x$ into a multiplication node before an immediately following power is applied. With $x = 2$:

```
3x^2    -> (3*x)^2 -> 36
3*x^2   -> 3*(x^2) -> 12
```

This behavior was checked against the executable. For an unambiguous polynomial coefficient, write $3x^2$. The declaration `%prec '*'` on the implicit-multiplication rule does not make this special production behave identically to an explicit multiplication in every context.

Chapter 4

How sqrt works, from text to answer

4.1 Step 1: recognize the tokens

For `sqrt(25)`, the lexer produces:

```
IDENTIFIER("sqrt")  '('  NUMBER(25)  ')'
```

There is no dedicated `SQRT` token. The identifier rule recognizes all names with the same pattern:

```
ID  [a-zA-Z_][a-zA-Z0-9_]*  
  
{ID} { yylval.sval = strdup(yytext); return IDENTIFIER; }
```

`yytext` contains the matched text. `strdup` copies it because the parser needs to retain the name after the lexer continues scanning. `yylval.sval` carries the string value to Bison.

4.2 Step 2: recognize a function call

The actual production in `src/parser.y` is:

```
| IDENTIFIER '(' arglist ')' {  
    $$ = ast_call($1, $3.items, $3.count);  
    free($1);  
}
```

Here `$1` is the function name. `$3.items` is an array of argument AST pointers; `$3.count` is the number of arguments. The parser recognizes the shape of a function call. It does not yet decide whether the function is supported.

4.3 Step 3: store the function-call node

The implementation in `src/ast.c` is:

```
ASTNode *ast_call(const char *name, ASTNode **args, int argc) {  
    ASTNode *n = alloc_node(N_CALL);  
    n->name = strdup(name);
```

```

    n->args = args;
    n->argc = argc;
    return n;
}

```

For this example the resulting structure is:

```

N_CALL
  name = "sqrt"
  argc = 1
  args[0] -> N_NUM, num = 25

```

The constructor copies the name, so the parser can safely free its original identifier string. Later, `ast_free` recursively frees the call's arguments, argument array, name, and node.

4.4 Step 4: dispatch to the function evaluator

In `src/eval.c`, the evaluator switches on the node kind:

```

case N_CALL:
    return call_function(n->name, n->args, n->argc, quiet, ok);

```

The name and argument list are passed to `call_function`. This helper recognizes supported names using `strcmp`. In C, `strcmp(a,b) == 0` means the strings are equal.

4.5 Step 5: check the argument count

After recognizing one of the supported one-argument functions, the helper does:

```

if (argc != 1) {
    semantic_error("'s' expects 1 argument, got %d", name, argc);
    *ok = 0;
    return 0;
}
double a = eval(args[0], quiet, ok);
if (!*ok) return 0;

```

`sqrt(25)` has one argument and passes. `sqrt(25,9)` has two arguments, so it produces a semantic error. Argument count is called **arity**. The function expects an expression as its argument, so `sqrt(9+16)` also works: the argument subtree is evaluated to 25 first.

4.6 Step 6: check the domain and call the C library

The following is the same square-root branch with its long error line expanded for readability:

```

if (strcmp(name, "sqrt") == 0) {
    if (a < 0) {
        if (!quiet)
            math_error("sqrt of negative number %g", a);
        *ok = 0;
    }
}

```

```
    return 0;
}
return sqrt(a);
}
```

1. Confirm that the name is `sqrt`.
2. Reject a negative real input.
3. Call C's `sqrt` function from `<math.h>`.
4. Return the result as a `double`.

For $a = 25$, the library returns 5. The calculator prints the result using `format_number` in `src/util.c`, so a whole-number result usually appears as 5, rather than 5.000000.

Say it simply: I implemented the language-level function handling: recognizing the call, storing the arguments, checking the count and domain, and dispatching it. The numerical square-root calculation is done by C's math library.

4.7 What the project does not implement inside `sqrt`

MathScript does not contain a Newton iteration or a hand-written square-root algorithm. If asked how the numerical library works internally, distinguish that separate topic from the code you wrote. Do not claim that a particular library algorithm appears in this repository.

Also, $\sqrt{25} = 5$ is the principal nonnegative square root. Solving $y^2 = 25$ is a different task and produces $y = -5$ and $y = 5$.

Chapter 5

Every supported function and related rule

5.1 Function reference

Function	Arguments	Meaning / checks	Example result
<code>sqrt(a)</code>	1	Nonnegative real square root; rejects $a < 0$	<code>sqrt(25)</code> : 5
<code>abs(a)</code>	1	Absolute value; calls C's <code>fabs</code>	<code>abs(-5)</code> : 5
<code>sin(a)</code>	1	Sine; input in radians	<code>sin(pi/2)</code> : about 1
<code>cos(a)</code>	1	Cosine; input in radians	<code>cos(0)</code> : 1
<code>tan(a)</code>	1	Tangent; input in radians	<code>tan(0)</code> : 0
<code>log(a)</code>	1	Natural logarithm; rejects $a \leq 0$	<code>log(e)</code> : about 1
<code>pow(a,b)</code>	2	Raise a to power b	<code>pow(2,3)</code> : 8

These functions are built in. The language does not currently allow users to define a new function body such as `f(y)=y*y`.

5.2 `abs`, `sin`, `cos`, `tan`, and `log`

The short dispatch branches are:

```
if (strcmp(name, "abs") == 0) return fabs(a);
if (strcmp(name, "sin") == 0) return sin(a);
if (strcmp(name, "cos") == 0) return cos(a);
if (strcmp(name, "tan") == 0) return tan(a);
```

`fabs` handles floating-point absolute values. C's integer `abs` would not be the correct choice for general `double` inputs. For example, `abs(-2.5)` should produce 2.5.

Trigonometric input is in radians, not degrees. Since $180^\circ = \pi$ radians, $90^\circ = \pi/2$. That is why `sin(pi/2)` is a useful demonstration. The code does not explicitly detect tangent's singularities; floating-point approximations near $\pi/2$ may give a very large number.

For logarithm, the helper checks `a <= 0` before calling `log(a)`. The base is e , so `log(e) = 1` here. There is no separate built-in `log10` in this implementation.

5.3 pow: evaluate two arguments

```
if (strcmp(name, "pow") == 0) {
    if (argc != 2) {
        semantic_error("'pow' expects 2 arguments, got %d", argc);
        *ok = 0; return 0;
    }
    double a = eval(args[0], quiet, ok);
    if (!*ok) return 0;
    double b = eval(args[1], quiet, ok);
    if (!*ok) return 0;
    return pow(a, b);
}
```

For `pow(2+1,2)`, the first argument becomes 3, the second becomes 2, and the result is $3^2 = 9$. For `pow(2,3^2)`, the second argument becomes 9, so the result is 512.

The power operator uses the same C library function:

```
case '^': return pow(a, b);
```

In MathScript, `^` means exponentiation. In C source, the `^` operator means bitwise XOR; therefore the evaluator and C generator translate MathScript power to a call to `pow`.

Current limitation: unlike `sqrt` and `log`, the power branches do not explicitly check every domain or overflow failure. Some inputs can produce NaN or infinity. Do not say that all math-library errors are converted into a MathScript error.

5.4 How argument lists work

```
arglist : /* empty */
        | expr
        | arglist ',' expr
        ;
```

This abbreviated rule shows the shapes. The actual actions allocate an array for the first argument and grow it with `realloc` when another argument is appended. The list contains AST pointers, not evaluated values. That is why nested expressions remain available for later evaluation.

An empty list means `sqrt()` can be parsed, but it fails the one-argument check during evaluation. The current list grammar also permits an initial comma, as in `sqrt(,25)`, because the list before that comma can be empty. That is a grammar limitation, not a feature to use in the demonstration.

5.5 Numbers, variables, and constants

```
case N_NUM:
    return n->num;

case N_VAR: {
    double v;
    if (!symtab_lookup(n->name, &v)) {
```

```
        semantic_error("Undefined variable '%s'", n->name);
        *ok = 0;
        return 0;
    }
    return v;
}
```

A number leaf already stores its value. A variable leaf stores a name; the evaluator looks up that name in the symbol table.

```
/calc
x = 25
sqrt(x)
```

The assignment stores 25. The later variable lookup returns 25, and the square-root call returns 5. The table initially registers `pi` and `e`. Although they are marked as constants, the current calculator assignment path does not enforce their protection; avoid claiming that reassignment is already blocked.

The number lexer accepts digits with an optional decimal part, such as 25 and 2.5. A leading minus is parsed separately as unary minus. Scientific notation and leading-dot decimals are not supported by this numeric token pattern.

Chapter 6

Recursion, errors, and code generation

6.1 One complete nested example

Consider `sqrt(25) + abs(-5)`. Its structure is:

```
BINOP '+'  
  CALL sqrt/1  
    NUM 25  
  CALL abs/1  
    UMINUS  
    NUM 5
```

1. `eval` sees addition and evaluates its left child.
2. That child calls `sqrt`; its argument evaluates to 25; it returns 5.
3. `eval` then evaluates the addition's right child.
4. That child calls `abs`; its argument negates 5 to obtain -5 .
5. `fabs(-5)` returns 5.
6. The addition returns $5 + 5 = 10$.

The same `eval` function calls itself on smaller subtrees. This is recursion. Literal numbers are the simplest stopping cases.

6.2 Why `ok` is a pointer

The caller starts with `int ok = 1` and passes `&ok`. The evaluator can write `*ok = 0` to tell all enclosing calls that something failed. The numeric return value alone is insufficient: zero can be a valid result or a placeholder returned after an error.

For `sqrt(-1) + 10`, the left subtree marks failure. The addition checks `if (!*ok) return 0;` and stops; it does not print a valid answer of 10. `src/calc.c` checks the success flag before printing TAC and a result.

6.3 Why quiet exists

`quiet=0` allows the evaluator to print mathematical errors. `quiet=1` suppresses those messages while still marking the sample invalid through `ok`. Graphing uses quiet evaluation because a function may be undefined at many sampled coordinates. Semantic errors such as an unknown function still print: quiet does not suppress them.

6.4 The four error classes

Class	Example	What is wrong?
Lexical	10 @ 5	The lexer cannot recognize @.
Syntax	10 + * 5	Recognized tokens do not form a valid expression.
Semantic	foo(5)	Valid call syntax, unsupported function name.
Semantic	sqrt(1,2)	Supported function, wrong argument count.
Mathematical	sqrt(-1)	Invalid real domain for square root.
Mathematical	10/0	Division by zero.

For a missing closing parenthesis, such as `sqrt(25`, the parser reports a syntax error before the evaluator runs.

6.5 Assignment is different from equality

`=` is a statement-level separator in the grammar. In calculator mode, `x = 5` is assignment and the left side must be a variable. In equation mode, `x^2 = 4` is an equation to solve. `==` is an expression comparison that returns 1 or 0. The evaluator rejects an assignment-style `=` node inside an expression.

6.6 Square root versus imaginary equation roots

The ordinary evaluator returns real `double` values, so `sqrt(-1)` is a mathematical error in calculator mode. Separately, `src/poly.c` represents real and imaginary parts for polynomial roots. Thus `y^2 + 1 = 0` in equation mode can print $0 - i$ and $0 + i$. The polynomial solver's complex-root support does not make every calculator function a complex-valued function.

6.7 What gen demonstrates

```
/calc
gen sqrt(25) + abs(-5)
```

The calculator first evaluates the AST normally. If no error was reported, `codegen_run` writes `output/generated.c`, invokes the system C compiler, then executes the generated program. A representative generated body is:

```
double t1 = sqrt(25);
double t2 = -(5);
double t3 = fabs(t2);
```

```
double t4 = t1 + t3;  
double result = t4;  
printf("%g\n", result);
```

The generated program includes `<math.h>` and the build command uses `-lm` to link the math library where required. A header provides function declarations; linking supplies the implementation. Note again: this generator walks the AST directly, rather than parsing the printed TAC.

Chapter 7

A demonstration you can rehearse

7.1 A five-minute sequence

Run these commands from the MathScript project folder:

```
./build.sh
./build/mathscript
```

Inside the program, use:

```
/precedence
/calc
2 + 3 * 4
(2 + 3) * 4
-2^2
2^3^2
sqrt(9 + 16)
sqrt(25) + abs(-5)
pow(2,3)
sin(pi/2)
log(e)
sqrt(-1)
sqrt(1,2)
foo(5)
gen sqrt(25) + abs(-5)
exit
/exit
```

Keep TAC enabled for the precedence examples. Point at the temporary that computes multiplication or the inner exponent. Then open `src/parser.y` and show the five declarations. For functions, open `src/eval.c` and show the argument-count check, recursive argument evaluation, and square-root branch.

7.2 Expected answers for the successful calculations

Expression	Answer
$2 + 3 * 4$	14
$(2 + 3) * 4$	20
-2^2	-4
2^{3^2}	512
$\text{sqrt}(9 + 16)$	5
$\text{sqrt}(25) + \text{abs}(-5)$	10
$\text{pow}(2,3)$	8
$\sin(\pi/2)$	approximately 1
$\log(e)$	approximately 1

The three invalid calls demonstrate a mathematical error, an arity semantic error, and an unknown-function semantic error, respectively.

7.3 A quick practice exercise

Before running them, predict:

1. $10 - 3 - 2$
2. $20 / 5 * 2$
3. $(-2)^2$
4. 2^{-3}
5. $\text{sqrt}(\text{pow}(3,2) + \text{pow}(4,2))$
6. $\text{pow}(2,3^2)$
7. $\text{abs}(-2.5)$

Answers: 5, 8, 4, 0.125, 5, 512, and 2.5. For each answer, explain the tree grouping before describing the arithmetic.

Chapter 8

Short viva questions and answers

1. Where is operator precedence implemented?

In `src/parser.y`, using Bison's `%left`, `%right`, and `%prec` declarations. They influence the generated parser tables.

2. Does the lexer decide multiplication happens first?

No. It returns tokens. The parser resolves the expression structure using the grammar and precedence declarations.

3. What is the difference between precedence and associativity?

Precedence chooses how different precedence levels group. Associativity chooses grouping at the same level, such as repeated subtraction or power.

4. Why is exponentiation right associative?

The grammar declares `%right '^'`. Therefore 2^3^2 groups as $2^{(3^2)}$.

5. Why does minus two squared give minus four?

Power has higher precedence than the unary-minus production. The AST negates the result of 2^2 . Parenthesizing the negative number gives a different tree.

6. Does Bison calculate the expression?

Our Bison actions create AST nodes. Our C evaluator later calculates their values. Bison could run other semantic actions, but these actions build trees.

7. Is `sqrt` a keyword in the lexer?

No. It is an identifier followed by parentheses and an argument list. The evaluator recognizes the supported function name.

8. How can sqrt accept an expression instead of a number?

Each argument is an expression AST. The function helper recursively evaluates that AST before calling the numerical library function.

9. What does strcmp returning zero mean?

The two strings are equal. It is how the dispatcher matches a stored function name.

10. Why check argc?

The parser accepts a general argument list, but each built-in function has a required arity. Square root expects one argument; power expects two.

11. Why not just return zero on an error?

Zero is also a valid numerical result. The shared success flag distinguishes failure from a successful zero.

12. Did you implement the numerical square-root algorithm?

No. I implemented language parsing, AST construction, checks, and dispatch. C's math library computes the numerical square root.

13. Why use fabs for abs?

MathScript uses floating-point values. `fabs` accepts and returns floating-point values, unlike the integer absolute-value function.

14. Are sin and cos in degrees?

No, radians. Use `pi/2` for ninety degrees.

15. What base does log use?

The natural base e , because the implementation calls C's `log`.

16. Is pow(y,2) the same operation as y squared?

Yes. The parser creates different node shapes (call versus binary power), but both evaluator paths call the C power function after evaluating operands.

17. Why is foo(5) a semantic error instead of a syntax error?

Its shape is a valid function call. Its name is not supported by the evaluator.

18. How do you prove precedence in the running program?

Show both the numeric answer and the TAC operation order, then identify the matching declarations in `src/parser.y`.

19. What happens to the AST afterwards?

The driver frees it after the code backend finishes. `ast_free` recursively releases children and owned strings and arrays.

20. What would you do to add another built-in function?

For a one-argument function such as a future `exp`, add its name to the recognized group, add a dispatch branch and any required domain checks, and add tests. The general function-call grammar already accepts its syntax. Also check the C-generation mapping and behavior in graph/equation contexts.

21. What is one honest limitation to mention?

Implicit multiplication followed by power has a grouping limitation; write explicit multiplication for coefficients. Power errors are also not checked as completely as square-root and logarithm domain errors.

The two answers to memorize

Say it simply: Operator precedence is implemented in the Bison parser. Later declarations bind more tightly. Associativity decides grouping at the same level. The parser constructs the correct AST, and the evaluator follows that tree.

Say it simply: A function call is parsed as an identifier with argument expressions. The evaluator checks the function name and argument count, evaluates the arguments, checks the required domain, and calls the matching C math function. For square root, a negative argument is rejected before calling `sqrt`.

Appendix A

The actual evaluator source

This is a self-contained snapshot of `src/eval.c`. It is included here so the guide can be read without access to the project directory. The shorter listings in the chapters explain individual parts of this implementation.

```
#include <math.h>
#include <string.h>
#include "eval.h"
#include "symtab.h"
#include "errors.h"

static double call_function(const char *name, ASTNode **args, int argc,
                           int quiet, int *ok) {
    /* one argument functions */
    if (strcmp(name, "sqrt") == 0 || strcmp(name, "abs") == 0 ||
        strcmp(name, "sin") == 0 || strcmp(name, "cos") == 0 ||
        strcmp(name, "tan") == 0 || strcmp(name, "log") == 0) {
        if (argc != 1) {
            semantic_error("'s' expects 1 argument, got %d", name, argc);
            *ok = 0; return 0;
        }
        double a = eval(args[0], quiet, ok);
        if (!*ok) return 0;

        if (strcmp(name, "sqrt") == 0) {
            if (a < 0) { if (!quiet) math_error("sqrt of negative number %g", a); *ok =
0; return 0; }
            return sqrt(a);
        }
        if (strcmp(name, "abs") == 0) return fabs(a);
        if (strcmp(name, "sin") == 0) return sin(a);
        if (strcmp(name, "cos") == 0) return cos(a);
        if (strcmp(name, "tan") == 0) return tan(a);
        /* log */
        if (a <= 0) { if (!quiet) math_error("log of non-positive number %g", a); *ok =
0; return 0; }
        return log(a);
    }

    /* two argument functions */
    if (strcmp(name, "pow") == 0) {
        if (argc != 2) {
            semantic_error("'pow' expects 2 arguments, got %d", argc);
```

```

        *ok = 0; return 0;
    }
    double a = eval(args[0], quiet, ok);
    if (!*ok) return 0;
    double b = eval(args[1], quiet, ok);
    if (!*ok) return 0;
    return pow(a, b);
}

semantic_error("Unknown function '%s'", name);
*ok = 0;
return 0;
}

double eval(ASTNode *n, int quiet, int *ok) {
    if (!*ok || !n) return 0;

    switch (n->kind) {
        case N_NUM:
            return n->num;

        case N_VAR: {
            double v;
            if (!symtab_lookup(n->name, &v)) {
                semantic_error("Undefined variable '%s'", n->name);
                *ok = 0;
                return 0;
            }
            return v;
        }

        case N_UMINUS:
            return -eval(n->left, quiet, ok);

        case N_CALL:
            return call_function(n->name, n->args, n->argc, quiet, ok);

        case N_BINOP: {
            double a = eval(n->left, quiet, ok);
            if (!*ok) return 0;
            double b = eval(n->right, quiet, ok);
            if (!*ok) return 0;

            switch (n->op) {
                case '+': return a + b;
                case '-': return a - b;
                case '*': return a * b;
                case '/':
                    if (b == 0) { if (!quiet) math_error("Division by zero"); *ok = 0;
return 0; }
                    return a / b;
                case '^': return pow(a, b);
                case '<': return a < b ? 1 : 0;
                case '>': return a > b ? 1 : 0;
                case OP_GE: return a >= b ? 1 : 0;
                case OP_LE: return a <= b ? 1 : 0;
                case OP_EQ: return a == b ? 1 : 0;
                case OP_NE: return a != b ? 1 : 0;
                case '=':

```

```

        /* '=' is only meaningful to the mode-specific
           backends (assignment / equation), never here. */
        semantic_error("'=' cannot appear inside an expression");
        *ok = 0;
        return 0;
    }
}
*ok = 0;
return 0;
}

```

Appendix B

Source-file revision map

Use this map to find the code during your presentation. Search for the function or declaration named here rather than relying on line numbers, which can change after edits.

File	Search for
src/lexer.l	NUMBER, ID, yylval
src/parser.y	%left, UMINUS, arglist
src/ast.h	NodeKind, ASTNode
src/ast.c	ast_call, ast_binop, ast_free
src/eval.c	call_function, eval
src/calc.c	calc_run
src/errors.c	semantic_error, math_error
src/symtab.c	symtab_init, symtab_lookup
src/ir.c	tac_build
src/codegen.c	codegen_expr, codegen_run
src/main.c	parse_line, process_line
tests/run_tests.sh	Precedence, function, and error test cases

Compiling this guide

This file contains its own code listings and does not require the MathScript source files when compiling. With a LaTeX distribution installed, compile twice to populate the table of contents:

```
pdflatex QNA.tex
pdflatex QNA.tex
```

You can also upload the file to a LaTeX editor that provides pdfLaTeX.